Software Requirements Analysis

CBC Broadcast Scheduling Web Application

COMP3059
Group 7

Riley Thomas - 101505655
Andrew Sas - 101462946
Shaffaq Hai - 101500534
Oskar Molenda - 101460691

1.0 Introduction **3**
1.1 Purpose 3
1.2 Scope 3
2.0 System Overview **4**
2.1 Project Perspective 4
2.2 System Context 4
2.3 General Constraints 5
2.4 Assumptions and Dependencies 5
3.0 Functional Requirements **6**
3.1 Features 6
3.2 Use Cases 8
3.2.1 Use Case #1 8
3.2.2 Use Case #2 9
3.3 Data Modeling and Analysis 10
3.3.1 Normalized Data Model Diagram: 10
3.3.2 Activity Diagram: 10
3.3.3 UML Class Diagram: 11
3.3.4 Sequence Diagram: 11
3.4 Process Modeling 12
3.4.1 Data Flow Diagram 12
4.0 Non-Functional Requirements **13**
5.0 Logical Database Requirements **14**
6.0 Other Requirements **15**
7.0 Approval **15**

1.0 Introduction

This document serves to outline the technological requirements for the CBC Broadcast Scheduling Web Application (henceforth BSWA)

1.1 Purpose

The purpose of the BSWA is to create a web application for CBC to help facilitate the processing of third-party schedule data into a balanced, formatted alignment document. The software product is intended to make a tedious manual process done through spreadsheets, into a simple semi-automated workflow with the aid of a visual interface.

1.2 Scope

The scope of this project is constrained to collecting, formatting, and manipulating scheduling data. The project aims to improve efficiency in the alignment creation process. After analysis of the current scheduling workflows, the project will design and develop a tool to automate and simplify data entry and manipulation for scheduling tasks. The tool first and foremost will look to integrate with the current scheduling paradigm that is used at CBC to create Para/Olympic alignment documents. The software will have tangible benefits of using a visual interface to facilitate easier data manipulation. The tool will not offer individual user customization of the system. The software will also aim to implement checks to make sure that the broadcast schedules will not be conflicting and alert the user of potential conflicts. CBC BSWA has no plans to offer any infrastructure changes to CBC and does not offer any post-deployment support and maintenance.

2.0 System Overview

2.1 Project Perspective

The BSWA will be a new standalone, browser-based application designed to ingest structured scheduling data as provided by the CBC. Once imported, technicians will make use of the system to organize and manipulate this data, then export it as a deliverable to be used in CBC scheduling processes.

2.2 System Context

The BSWA is a digital solution that integrates with the existing CBC scheduling workflow, aimed at improving efficiency and coordination during complex scheduling periods - specifically during events such as the Olympic and Paralympic Games. The BSWA provides a semi-automated system to assist technicians in optimizing time allocation, and clearing scheduling conflicts.

2.3 General Constraints

The functionality of the BSWA is reliant on the consistent format and structure of input data. The data that will be input must be pre-formated in the same tabular format and any data that is formatted incorrectly will reduce the efficacy of the software and result in loss efficiency or program errors. Additionally, the application will not include a method of cleaning or reformatting raw input data, which is assumed to be completed by the third-party provider.

2.4 Assumptions and Dependencies

The following assumptions have been made about the development of the CBC BSWA:

- CBC is not actively creating their own solution
- CBC is not collaborating with a third party to develop this solution
- The input data will remain in a consistent format
- The CBC has sufficient resources and infrastructure to host and operate the web application without acquiring additional or specialized hardware

3.0 Functional Requirements

3.1 Features

1. Interactive Web Application that enables employees to modify schedule data
   Inputs:
   - User input: Mouse clicks, drag and drop interactions, etc.
   - Schedule data: The schedule dataset is loaded and displayed in the interface.

   Processing:
   - Data validation: Checks to ensure that modifications to the schedule do not violate fundamental constraints.
   - Interface rendering: Updates made by the technician are quickly reflected both visually and in memory.
   - Error Handling: Highlight fields or entries where modification attempts would result in conflicts.

   Outputs:
   - Interface: a graphical, and interactive display of the scheduling matrix.
   - Schedule data: The generated alignment document, to either be saved or exported.
2. Ability to save and and load the schedule data
   Inputs:
   - Saving: Menu selection from technician
   - Loading: Menu selection from technician

   Processing:
   - Saving: In-memory schedule is restructured to JSON.
   - Loading: Database is queried to fetch schedule data, and read it into system usable format.

   Outputs:
   - DB Save: A new or updated schedule is persisted to the database.
   - Confirmation: Technician is notified that a schedule was successfully saved or loaded.
   - Display: Loaded schedule is populated graphically into the application.

3. Timeslot conflict resolution
   Inputs:
   - Schedule data: Imported and parsed scheduling data, typically JSON/BSON, containing show titles, start tiles, end times, assigned channels

   Processing:
   - Conflict detection: System iterates through input data, comparing times and preventing conflicts for the same time range in a given channel.
   - Overlap calculation: Conflicts related to partial scheduling overlaps are flagged.

   Outputs:
   - Visualization: All scheduling is mapped visually, along with visual indication of errors and issues.
   - Database update: All changes, automatic and manual are confirmed and persisted.

4. Automatic load balancing
   Inputs:
   - Unbalanced data: Validated scheduling data is loaded by technicians.
   - Balancing parameters: Parameters are provided to set balancing conditions.

   Processing:
   - Data ingestion: The system receives the schedule and balance constraints.
   - Algorithm execution: The balancing algorithm attempts to fit all programming into the available schedule matrix, optimally filling channels/nodes for maximum utilization within thresholds.

   Outputs:
   - Visualization: A visual diagram of utilization across all channels and nodes it displayed.
   - Database update: An updated version of the schedule complete with channel and node assignments is persisted to the database.

3.2 Use Cases

3.2.1 Use Case #1

Primary Actor: CBC Technician
Secondary Actors: BSWA System, Third-Party Data Providers
Description: CBC needs to transcribe third party input schedule data into a usable format that has no time conflicts.
Preconditions: System is accessible and operational, schedule data is provided.
Success Scenario:

1. Technician navigates to web application
2. Technician chooses to import schedule - selecting either spreadsheet or csv source
3. System parses and validates data
4. System populates scheduling window with ingested data, flagging complicated timeslot conflicts
5. Technician reviews alignment document, resolving issues
6. System stores validated alignment in database
7. Technician is notified of successful import

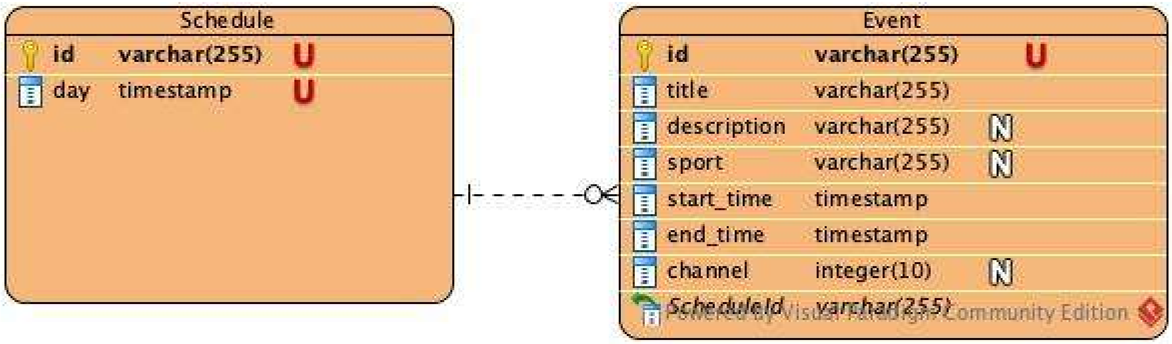
3.2.2 Use Case #2

Primary Actor: CBC Technician
Secondary Actors: BSWA System, BSWA Database
Description: CBC needs to take the input schedule and appropriately load balance the schedule material between all 16 channels, and 8 nodes, based upon parameters that may include channel availability, or content priority.
Preconditions: Valid schedule data has been provided with timeslot fields.
Success Scenario:

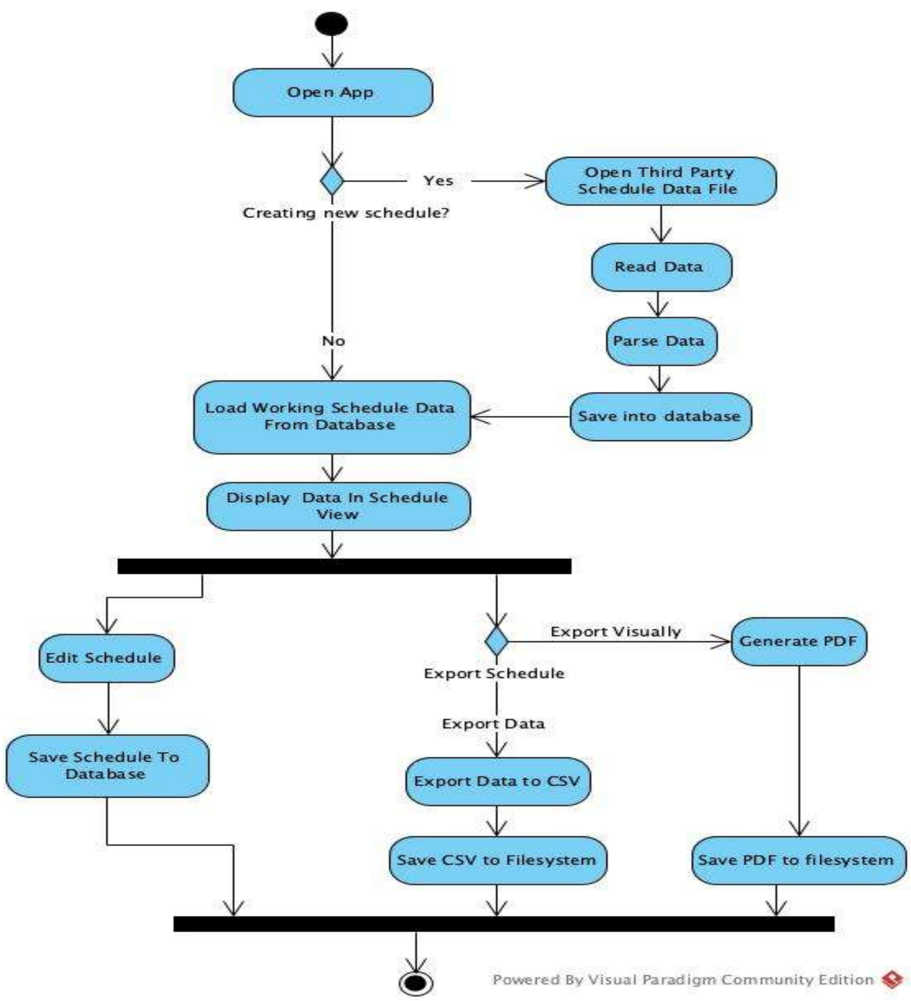
1. Technician selects load balancing option in menu
2. Technician selects a schedule dataset to balance, which is loaded from database
3. System automatically attempts to balance programming
4. System displays a visual rendition of channel and time utilization
5. Technician is prompted to review and make adjustments to generated alignment if necessary
6. Technician confirms schedule, and persists to database

3.3 Data Modeling and Analysis

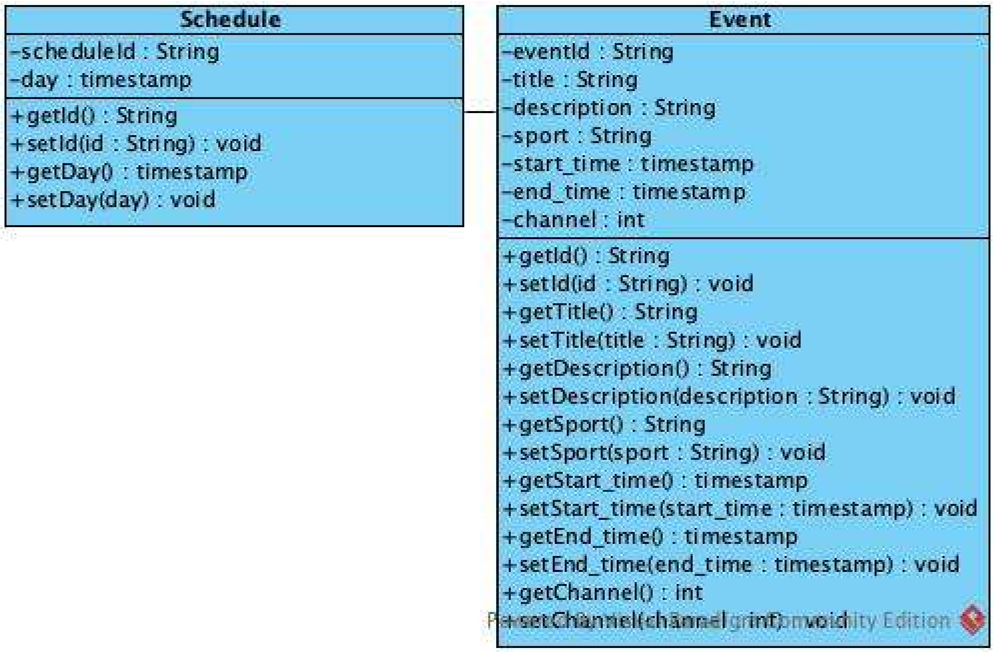
3.3.1 Normalized Data Model Diagram:



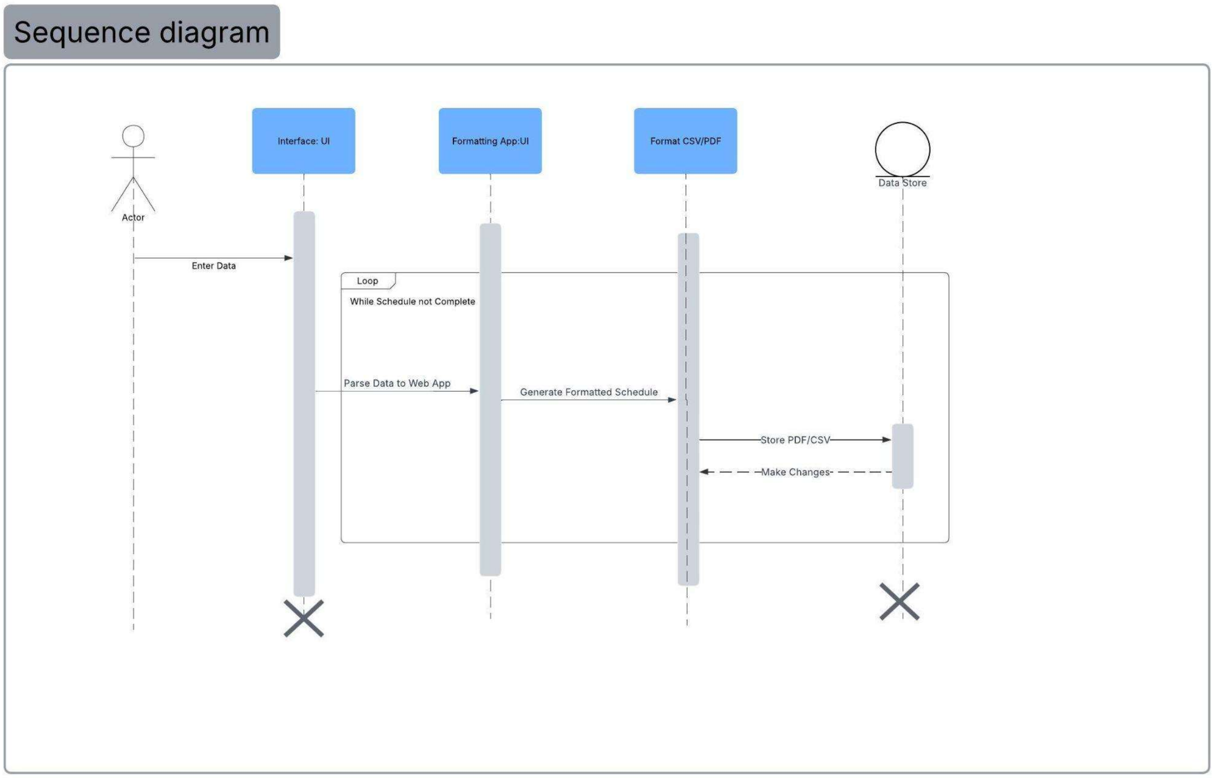
3.3.2 Activity Diagram:



3.3.3 UML Class Diagram:

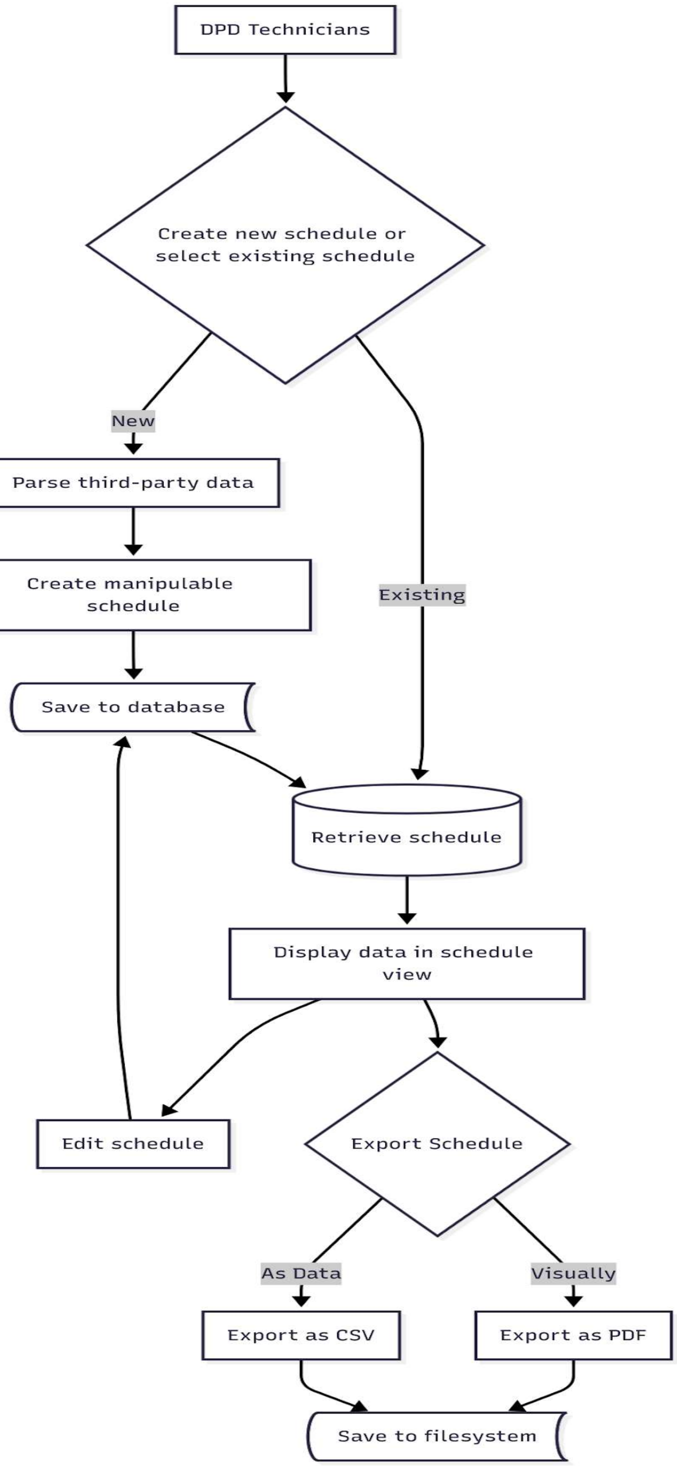


3.3.4 Sequence Diagram:



3.4 Process Modeling

3.4.1 Data Flow Diagram



4.0 Non-Functional Requirements

- 95% of all database transactions shall complete in under 2 seconds
- The system shall work on all major browsers released in the last 2 years
- Error logs shall record all system exceptions and be retained for a minimum of 30 days for debugging and audit purposes

5.0 Logical Database Requirements

The database will be using MongoDB. Storage requirements are minimal for the database, as the use of the system is for special events, which do not occur often, furthermore, the schedules themselves have a small data footprint. The schedules will be stored in the database in JSON/BSON format for easy retrieval / insertion to / from the database. Due to the short term usage of the system, long term storage of working schedule data is a low priority for the CBC, and thus extensive backup solutions are not required.

6.0 Other Requirements

Documentation:

Comprehensive documentation will be provided, including a User Guide for CBC technicians detailing use of the system, as well as a System Design Reference to support potential future expansion or modification of the system.

Testing:

The system will undergo both unit testing and integration testing to ensure that individual components function correctly and that the overall system performs as intended.

Logging:

Include logging in the system for use by system administrators to track usage of the system, as well as for use in finding potential errors post deployment.

Deployment:

The CBC BSWA will be handed over as a containerized docker image, to be deployed in the fashion that the CBC sees fit.

7.0 Approval

Project Role	Name	Signature	Date
Project Lead	Riley Thomas	RT	06/11/2025
Lead Engineer	Andrew Sas	A.S	06/11/2025
Engineer	Shaffaq Hai	s.h	06/11/2025
Engineer	Oskar Molenda	o.m	06/11/2025